

Design and Analysis of Algorithm

Lecture 22: Queue

Content



- **Operations on Queue**
- **Types of Queue**

Queue : Overflow and Underflow Conditions

A queue may have a limited space depending on the implementation. We must implement check conditions to see if we are not adding or deleting elements more than it can maximum support

The underflow condition checks if there exists any item

```
if (front == rear)
```

The overflow condition checks if the queue is full

```
if (rear == SIZE-1)
```

Queue : Declaration

Array

```
#define MAX 100

typedef struct Queue {
    int arr[MAX];
    int front;
    int rear;
} Queue;
```

Linked List

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

Queue : Creation

Array

```
void create (Queue *q)
{
    queue->front = -1;
    queue->rear = -1;
}
```

Linked List

```
typedef struct Queue {
    Node *front, *rear;
} Queue;

Queue* createQueue()
{
    Queue* q =
    (Queue*)malloc(sizeof(Queue));
    q->front = q->rear = NULL;
    return q;
}
```

Queue : Enqueue Operation

Array

```
void enqueue(struct Queue* queue, int x) {  
    if (queue->rear < MAX - 1) {  
        queue->arr[++queue->rear] =  
x;  
    }  
}
```

Linked List

```
void enqueue(Queue* q, int new_data)  
{  
    Node* new_node =  
createNode(new_data);  
  
    if (q->rear == NULL) {  
        q->front = q->rear =  
new_node;  
        return;  
    }  
    q->rear->next = new_node;  
    q->rear = new_node;  
}
```

Queue : Dequeue Operation

Array

```
void dequeue(struct Queue* queue) {  
    if (queue->front == queue->rear) {  
        queue->front++;  
    }  
}
```

Linked List

```
void dequeue(Queue* q)  
{  
    if (q->front == NULL && q->rear == NULL) {  
        printf("Queue Underflow\n");  
        return;  
    }  
    Node* temp = q->front;  
    q->front = q->front->next;  
  
    if (q->front == NULL)  
        q->rear = NULL;  
  
    free(temp);  
}
```

Queue : Array Implementation

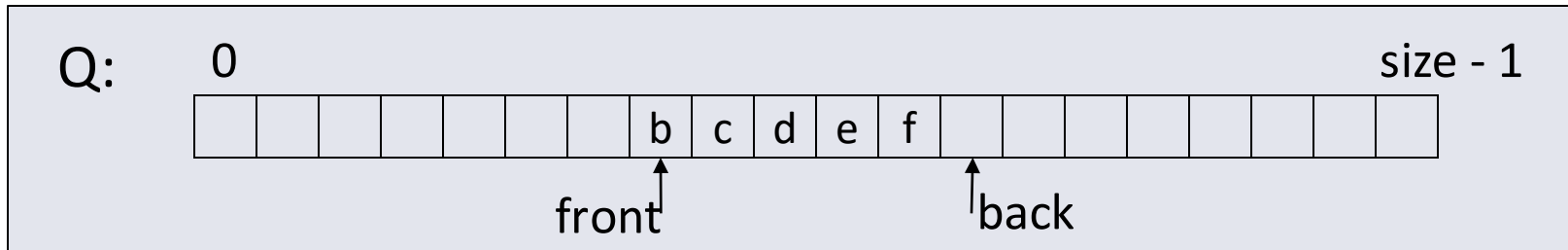
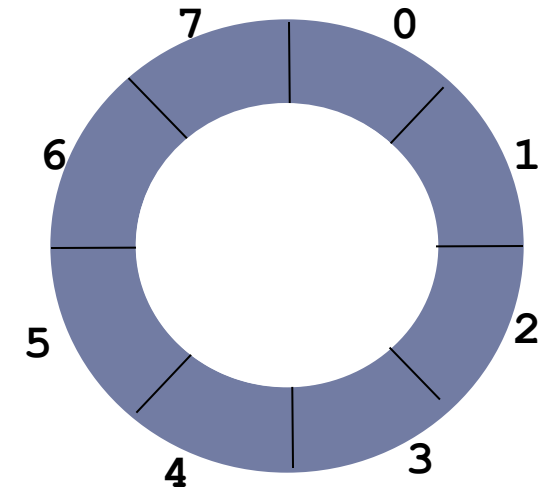
Keep track of the number of elements in the queue, size.

- Enqueue at the back of the array (size).
- Dequeue at the front of the array (index 0).

What is bad about this implementation?

Circular Queue : Array

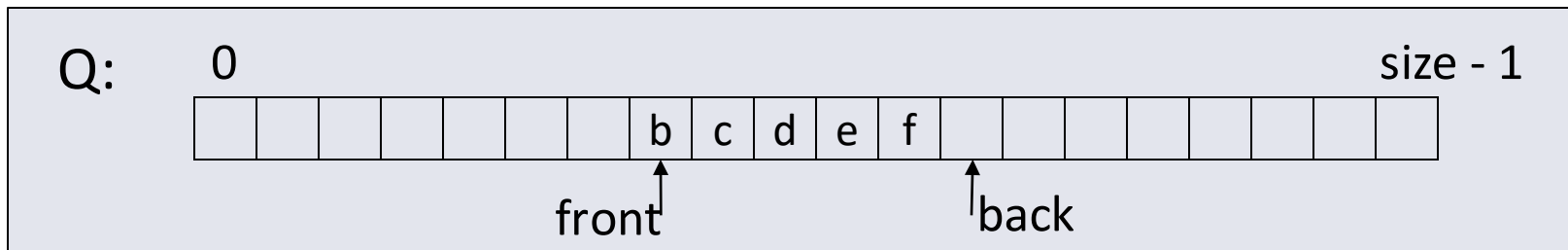
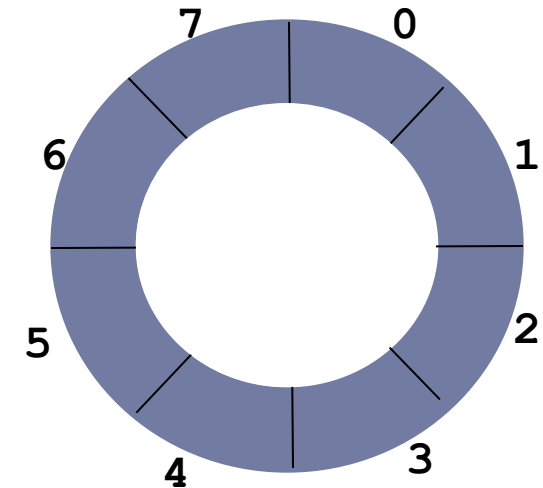
- ▶ Use a *circular array* to insert and remove items from a queue.
- ▶ The idea of a circular array is that the end of the array “wraps around” to the start of the array.



Circular Queue :Introduction

A circular queue is a fixed-size queue implemented using an array in which the last position is connected to the first to form a circle. It uses two pointers

- ▶ Front: Points to the first element in the queue.
- ▶ Rear: Points to the last inserted element.



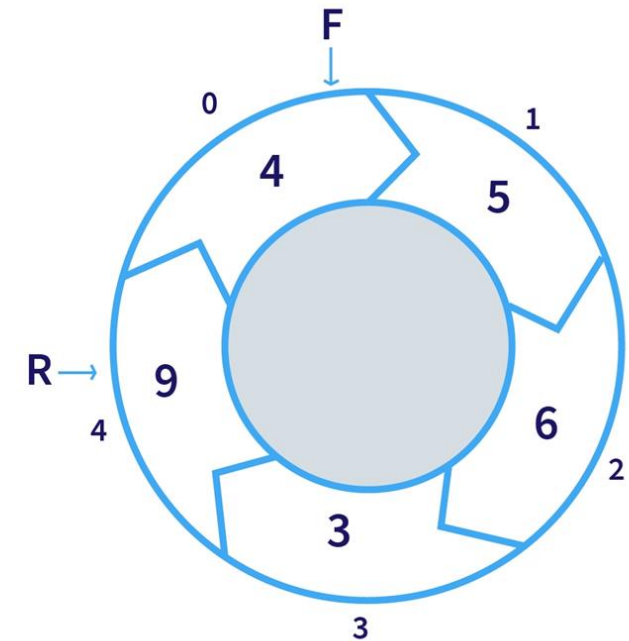
Circular Queue :Overflow Condition

This happens when the queue is full, and we try to insert another element.

There are two common ways to check for a full queue

(i) $(Rear + 1) \% Size == Front$

(ii) $count == size$



Circular Queue : Underflow Condition

This happens when the queue is empty, and we try to dequeue an element.

(i) $front == Rear$

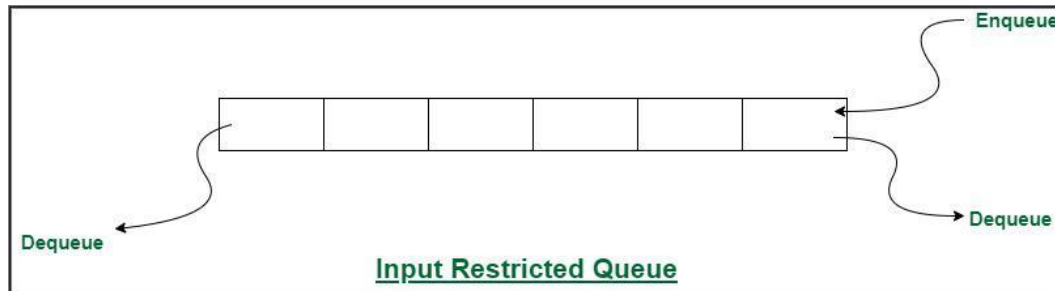
Other Type of Queues

Different types of queues are as follows:

1. Input Restricted Queue
2. Output Restricted Queue
3. Double Ended Queue
4. Ascending Priority Queue
5. Descending Priority Queue

Input Restricted Queue

In this kind of Queue , enqueue operation is allowed only at one end but dequeue operation can be performed at both end

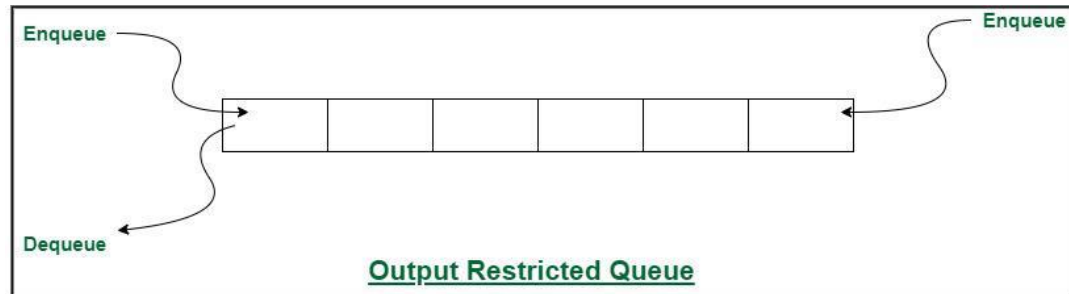


Example: Traffic Flow Management

Vehicles (elements) are allowed to enter (enqueue) only from the rear, while they can be dequeued (exit) from both the front (*green light opens*) or rear position (*emergency lane for ambulances or VIP vehicles*).

Output Restricted Queue

In this kind of Queue , dequeue operation is allowed only at one end but enqueue operation can be performed at both end



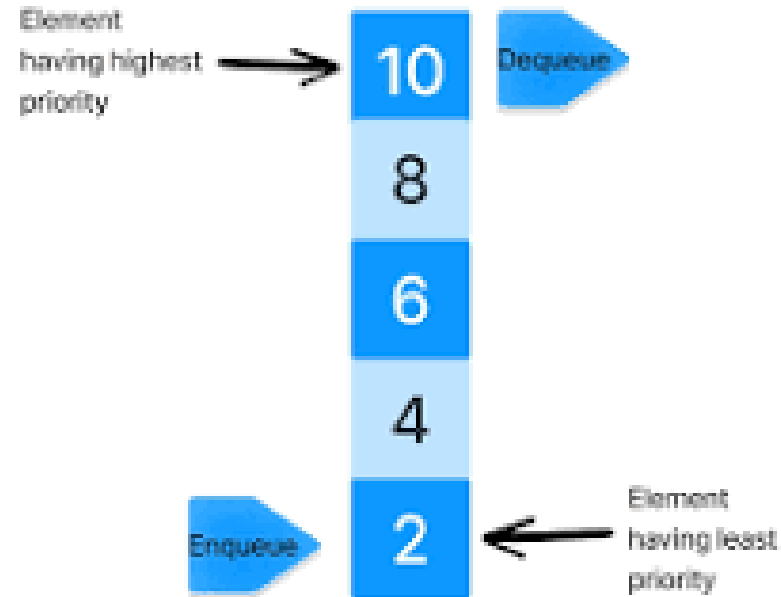
Example: Task Scheduling in Operating Systems

An Output Restricted Queue can be used in multi-level scheduling in operating systems, where high-priority tasks can be inserted at the front, and lower-priority tasks can be inserted at the rear. However, execution (dequeue) always happens in a strict order from the front

Priority Queue

A priority queue is a specialized queue where elements are processed based on their assigned priority, with higher-priority elements being served before lower-priority ones

The priority queue pattern enables a workload to process high-priority tasks more quickly than lower-priority tasks.



Priority Queue: Operations

1. **Insert (Enqueue):** This operation allows for adding a new element to the priority queue along with its priority value. The position of the element is determined by its priority, rather than its insertion order.
2. **Extract-Max / Extract-Min (Dequeue):** This operation removes and returns the element with the highest or lowest priority, depending on whether it's a max-priority or min-priority queue.
3. **Peek:** This operation returns the element with the highest or lowest priority without removing it from the queue, allowing you to check the next element to be processed.
4. **Increase / Decrease Key:** This allows adjusting the priority of an existing element, either by increasing or decreasing its priority, based on the queue type.

Priority Queue: Types

1. **Ascending Order Priority Queue:** numbers with smaller values are given higher priorities, i.e. the smallest element is at front of the queue.
2. **Descending Order Priority Queue:** numbers with larger values are given higher priorities, i.e. the largest element is at front of the queue.

Priority Queue: Example

